Lab 7: Fuzzing for Input Validation Bugs

Name: Sahajpreet Singh | SRN: PES1UG24CS397 | Section G

Application under test: **spam_scoring.py** (repo: L6_L7_33_spam_scoring, folder lab7) | Tool: Hypothesis + pytest

Summary

Function	Bug found by fuzzing	Failing input	Fix
sender_domain	IndexError (no '@'); AttributeError on None	'' , None	Type check; split with rpartition('@'); return '' if invalid
first_link	ValueError (no 'http' in text); AttributeError on None; returned bare 'http'	'' , None, 'http'	Type check; regex `https?://\S+`; return '' if no match
shout_ratio	ZeroDivisionError (no letters); TypeError on None	'' , None	Type check; return 0.0 when there are no letters

Result: before the fix **6 tests failed**; after the fix **6 tests passed**.

Step 1: Dependencies

Installed with `pip install hypothesis pytest` (Hypothesis 6.170.15, pytest 9.2.0, Python 3.13.5).

Step 2: Running the buggy application

Command: `python spam_scoring.py`, input `hello` (no '@'). The script crashes with an IndexError.

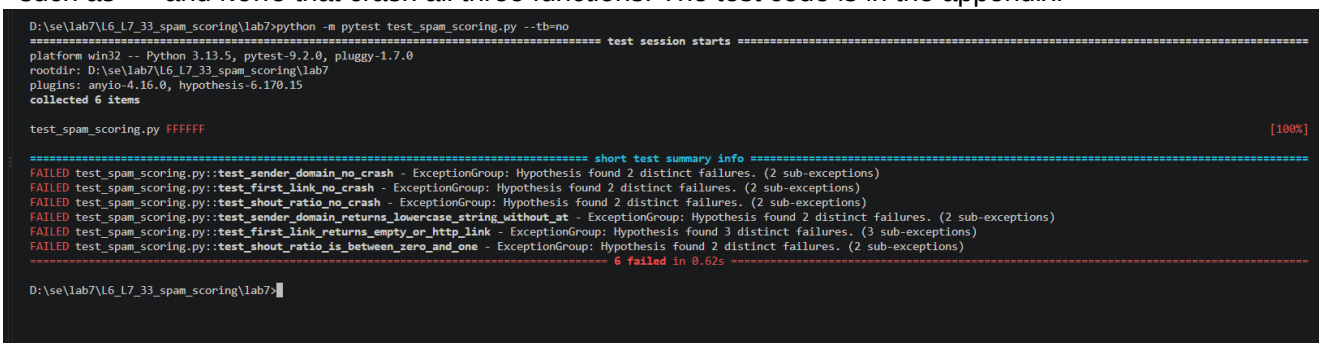
```
D:\se\lab7\L6_L7_33_spam_scoring\lab7>python spam_scoring.py
==== Running sender_domain ====
Enter a sender email (e.g. offers@Deals.com): hello
Traceback (most recent call last):
  File "D:\se\lab7\L6_L7_33_spam_scoring\lab7\spam_scoring.py", line 33, in <module>
    main()
    ~~~~^^
  File "D:\se\lab7\L6_L7_33_spam_scoring\lab7\spam_scoring.py", line 19, in main
    result = sender_domain(user_input)
  File "D:\se\lab7\L6_L7_33_spam_scoring\lab7\spam_scoring.py", line 2, in sender_domain
    return email.split('@')[1].lower()
           ~~~~~~~~~~~~~~~~~~~~^^^
IndexError: list index out of range

D:\se\lab7\L6_L7_33_spam_scoring\lab7>
```

Screenshot 1: buggy application crashes (IndexError: list index out of range)

Steps 3 and 4: Hypothesis tests failing on the buggy code

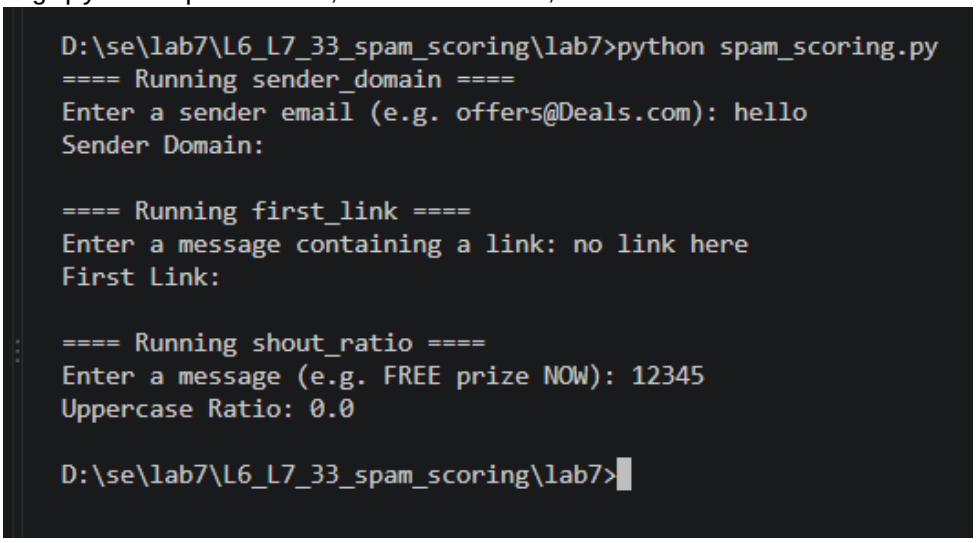
Command: `python -m pytest test_spam_scoring.py --tb=no`. Hypothesis generated inputs such as `''` and `None` that crash all three functions. The test code is in the appendix.



Screenshot 2: all 6 tests fail against the original script (6 failed)

Step 5: Fixed application

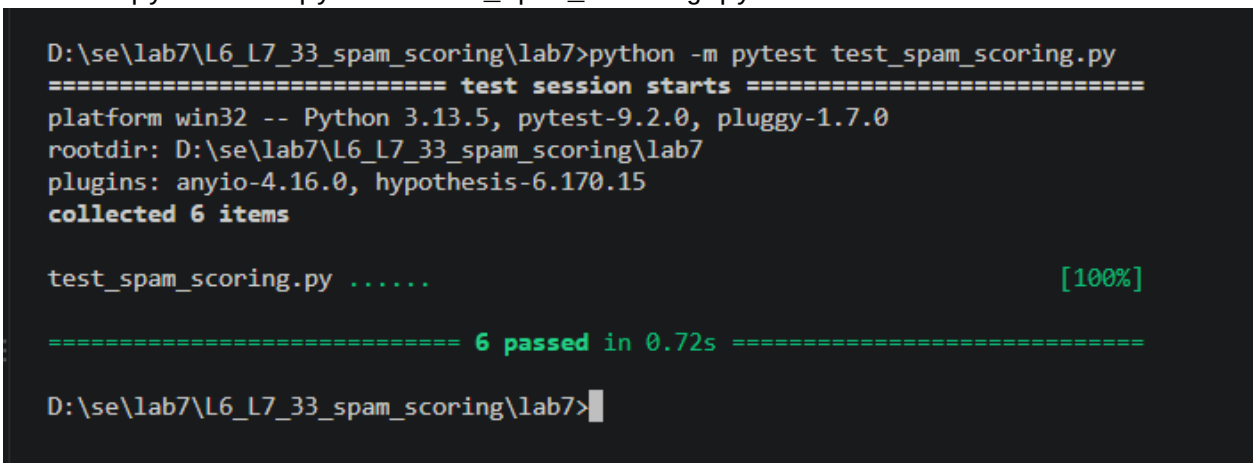
The three functions were hardened (see the fixed code in the appendix). Command: `python spam_scoring.py` with inputs `hello`, `no link here`, `12345`. No crash now.



Screenshot 3: fixed application handles the same bad inputs without crashing

Step 6: Tests re-run on the fixed code

Command: `python -m pytest test_spam_scoring.py`



Screenshot 4: all 6 tests pass (6 passed)

Appendix A: Fixed application (spam_scoring.py)

```
import re

_LINK_PATTERN = re.compile(r"https?://\S+", re.IGNORECASE)


def sender_domain(email):
    """Return the lower-cased domain of an email address, or "" if invalid."""
    if not isinstance(email, str):
        return ""
    local, at, domain = email.strip().rpartition("@")
    if not at or not local or not domain:
        return ""
    return domain.lower()


def first_link(text):
    """Return the first http(s) link in the text, or "" if there is none."""
    if not isinstance(text, str):
        return ""
    match = _LINK_PATTERN.search(text)
    return match.group(0) if match else ""


def shout_ratio(text):
    """Return the fraction of letters that are uppercase (0.0 if no letters)."""
    if not isinstance(text, str):
        return 0.0
    letters = [c for c in text if c.isalpha()]
    if not letters:
        return 0.0
    upper = [c for c in letters if c.isupper()]
    return len(upper) / len(letters)


def main():
    print("==== Running sender_domain ====")
    user_input = input("Enter a sender email (e.g. offers@Deals.com): ")
    result = sender_domain(user_input)
    print("Sender Domain:", result)

    print("\n==== Running first_link ====")
    user_input = input("Enter a message containing a link: ")
    result = first_link(user_input)
    print("First Link:", result)

    print("\n==== Running shout_ratio ====")
    user_input = input("Enter a message (e.g. FREE prize NOW): ")
    result = shout_ratio(user_input)
    print("Uppercase Ratio:", result)


if __name__ == "__main__":
    main()
```

Appendix B: Updated test file (test_spam_scoring.py)

```
from hypothesis import given, strategies as st
from spam_scoring import sender_domain, first_link, shout_ratio


@given(st.text() | st.none())
def test_sender_domain_no_crash(s):
    try:
        sender_domain(s)
    except Exception as e:
        assert False, f"sender_domain crashed with: {e!r} for input {s!r}"


@given(st.text() | st.none())
def test_first_link_no_crash(s):
    try:
        first_link(s)
    except Exception as e:
        assert False, f"first_link crashed with: {e!r} for input {s!r}"


@given(st.text() | st.none())
def test_shout_ratio_no_crash(s):
    try:
        shout_ratio(s)
    except Exception as e:
        assert False, f"shout_ratio crashed with: {e!r} for input {s!r}"


# ---------- extra property checks (return values stay sensible) ----------
@given(st.text() | st.none())
def test_sender_domain_returns_lowercase_string_without_at(s):
    result = sender_domain(s)
    assert isinstance(result, str)
    assert result == result.lower()
    assert "@" not in result


@given(st.text() | st.none())
def test_first_link_returns_empty_or_http_link(s):
    result = first_link(s)
    assert isinstance(result, str)
    assert result == "" or result.startswith(("http://", "https://"))


@given(st.text() | st.none())
def test_shout_ratio_is_between_zero_and_one(s):
    result = shout_ratio(s)
    assert isinstance(result, float)
    assert 0.0 <= result <= 1.0
```

Appendix C: Original buggy application (for reference)

```
def sender_domain(email):
    return email.split('@')[1].lower()

def first_link(text):
    start = text.index('http')
    end = text.find(' ', start)
    if end == -1:
        end = len(text)
    return text[start:end]

def shout_ratio(text):
    letters = [c for c in text if c.isalpha()]
    upper = [c for c in letters if c.isupper()]
    return len(upper) / len(letters)

def main():
    print("==== Running sender_domain ====")
    user_input = input("Enter a sender email (e.g. offers@Deals.com): ")
```

```
    result = sender_domain(user_input)
    print("Sender Domain:", result)

    print("\n==== Running first_link ====")
    user_input = input("Enter a message containing a link: ")
    result = first_link(user_input)
    print("First Link:", result)

    print("\n==== Running shout_ratio ====")
    user_input = input("Enter a message (e.g. FREE prize NOW): ")
    result = shout_ratio(user_input)
    print("Uppercase Ratio:", result)

if __name__ == "__main__":
    main()
```